

OP1	$1 * f(x) \rightarrow f(x)$	($f(x)$ is any real function of x)
OP2	$\int r * f(x) \rightarrow r \int f(x) dx$	(r is any real number)
OP3	$\int \sin(x) dx \rightarrow -\cos(x) dx$	
OP4	$\int \cos(x) dx \rightarrow \sin(x)$	
OP5	$\int [f_1(x) + f_2(x)] dx \rightarrow \int f_1(x) dx + \int f_2(x) dx$	
OP6	$\int u dv \rightarrow uv - \int v du$	(integration by parts)

Figure 19.3a Typical calculus operators.

$\int [\sin(x) + \cos(x)] dx$	$(\text{int}((+ \sin \cos) x))$
$\int \cos^2(x) dx$	$(\text{int}(\uparrow \cos 2) x))$
$\int x * e^x dx$	$(\text{int}((* \text{id}(\uparrow e \text{id})) x))$

Figure 19.3b Typical calculus representations.

the result obtained from the opposite bindings. The choice of the result obtained with u bound to f_1 and dv to f_2 or of an incorrect or poor operator at a given stage in the solution will lead to failure or possibly to a lengthy solution. The learning problem then is to create or refine heuristic rules which suggest when an operator should be used.

All heuristics in LEX are of the form

If: The integrand pattern is $P *$
 Then: Apply OP_n with bindings B .

For example, a typical heuristic for OP6 would be

$\int f(x) * \text{trig}(x) dx \rightarrow$ Apply OP6 with bindings $u = f(x)$, and $dv = \text{trig}(x) dx$

Part of the refinement problem is the generalization or specialization of the heuristics to apply to as many consistent instances as possible. Generalization and specialization are achieved in LEX through the use of a hierarchical description tree. A segment of this tree is depicted in Figure 19.4. Thus, when a rule applies to more than a single trig function such as to both sin and cos, the more general term trig would be substituted in the rule. Likewise, when a rule is found which applies to both log and exp functions, the exp_log description would be used.

LEX is comprised of four major components as illustrated in Figure 19.5. The Problem Generator selects and generates informative integration problems which are submitted to the Problem Solver for solution. The Generator was included as part of the system to provide well-ordered training examples and to make the system a fully automatic learner. The Problem Solver attempts to find a solution to this problem using available heuristics and operators. (A solution has been found when an operator produces an expression not containing an integral.)

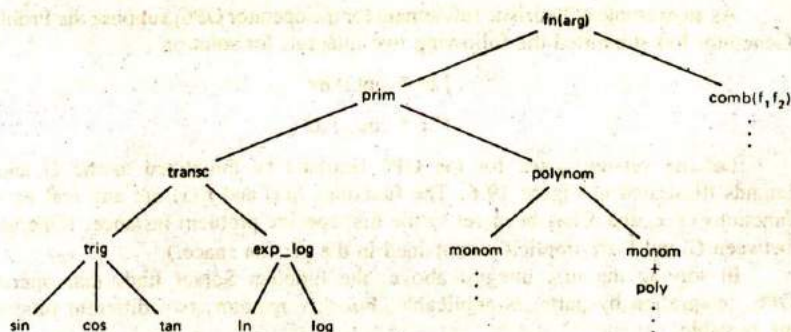


Figure 19.4 A segment of the LEX generalization tree grammar.

Output from the Problem Solver is some solution together with a complete trace of the solution search. This is presented to the Critic unit for evaluation. The Critic then analyzes the solution trace, comparing it to a least-cost path and passes related positive or negative training instances to the Generalizer. A positive instance is an operator which lies on the least-cost path, while a negative instance is one lying off the path. Given these examples, the Generalizer modifies heuristics to improve the selection of operators for best application during an attempted solution.

During the learning process each operator is given a *version space* of heuristic rules associated with it. Rules in the version space give the conditions under which the operator applies as well as the domain of states that can be produced. The version space is actually stored as two bounding rules, a rule G which gives the most general conditions of application and a rule S which gives the most specific conditions. Between these two bounds are implicitly contained all plausible versions of the heuristic. As the system learns, the bound S is made more general to include all positive instances presented by the Critic while the bound G is made more specific to exclude all negative instances. When the two bounds become equal ($G = S$), the correct heuristic has been learned.

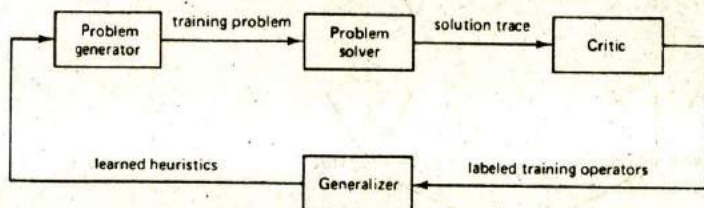


Figure 19.5 LEX learning model.

As an example of heuristic refinement for the operator OP6, suppose the Problem Generator has submitted the following two integrals for solution.

$$\int 2x * \sin(x) dx$$

$$\int 2x * \cos(x) dx$$

Let the version space for the OP6 heuristic be initialized to the G and S bounds illustrated in Figure 19.6. The functions $f_1(x)$ and $f_2(x)$ are any real-valued functions of x , and S has been set to the first specific problem instance. (Operators between G and S are implicitly contained in the version space.)

In solving the first integral above, the Problem Solver finds that operator OP6, integration by parts, is applicable. For this operator, two different bindings are possible

$$(a) \quad u = 2x \qquad (b) \quad u = \sin(x)$$

$$dv = \sin(x) dx \qquad dv = 2x dx$$

If the bindings given by a are used, OP6 produces the new expression

$$2x * (-\cos(x)) - \int 2 * (-\cos(x)) dx$$

which can be further reduced using OP2, OP4 and other simplification operators to give the correct solution

$$-2x * \cos(x) + 2 * \sin(x) + C$$

For this binding, the Critic will label this as a positive instance.

On the other hand, if the variable bindings given in b are used, OP6 produces the more complex expression

$$x^2 * \sin(x) - \int x^2 * \cos(x) dx$$

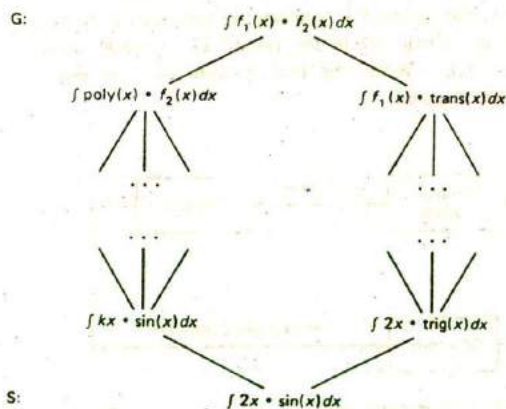


Figure 19.6 Version space with bounds G and S .

In this case the Critic will label that instance as a negative one. This negative instance will be used to adjust the version space to exclude this instance by specializing the G integrand to either

$$\text{poly}(x) * f_2(x) \, dx \quad \text{or to} \quad f_1(x) * \text{tran}(x) \, dx$$

with corresponding bindings

$$\begin{array}{ll} u = \text{poly}(x) & \text{and} \quad u = f_1(x) \\ dv = f_2(x) \, dx & dv = \text{tran}(x) \, dx \end{array}$$

both of which exclude the negative instance.

After the Problem Solver attempts to solve the second integral, $\int 2x * \cos(x) \, dx$, the Critic labels the binding $u = 2x$, $dv = \cos(x) \, dx$ as positive. However, this example is not yet included in the version space. Consequently, S must be generalized to include this instance. The bound S is generalized by finding the least general function in the description hierarchy which includes both $\sin$ and $\cos$ (Figure 19.4). This generalization produces the new version space

$$G: \int f_1(x) * \text{tran}(x) \, dx$$

$$S: \int 2x * \text{trig}(x) \, dx$$

Through repeated attempts at solving such well organized problems, LEX is able to create and refine heuristics for each operator which designate when that operator should be applied. Thus, the refined heuristics reduce the class of problems to those producing acceptable solutions.

19.4 THE INDUCE SYSTEM

Several versions of the INDUCE system were developed beginning in the late 1970s (Larson and Michalski, 1977, and Dietterich and Michalski, 1981). INDUCE is a system which discovers similar patterns among positive examples of objects and formulates generalized descriptions which characterize the class patterns. The description language and the internal representation used in the system are an extension of first order predicate calculus. This is one of the unique features of INDUCE.

Before outlining how the system operates, we introduce a few new terms:

Selector. A relational statement defined by either a predicate or a function together with the values it assumes. Typical selectors used to describe objects or events in INDUCE are [shape = square $\vee$ rectangular], [color = green], [size $\leq$ 4], [number_spots = 3 . . 8], [ontop(a,b)], and so on.

Complex. A logical product of selectors. A complex is a conjunctive description of some object (each selector in brackets is regarded as a conjunct). For example, a plant leaf might be represented as

```
Leaf_1 [contains(Leaf_1,spots)][color(Leaf_1) = yellow]&
[shape(Leaf_1 = curled)][length(Leaf_1) = 4][width(Leaf_1 = 2.3]
```

This expression states that a particular leaf contains spots, is yellow, curled at the sides, is 4 cm in length and 2.3 cm wide.

Cover. A cover of a set of events E is a set of complexes which describe or contain all events in E . A cover of the set E_1 against the set E_2 , where E_1 and E_2 are disjoint is a set of complexes which cover all events in E_1 and no event in E_2 . For example, the complex [shape = circle] [color = red] [size < 5] covers the set of all red circles of size less than 5 and none of the set of circles greater than 5.

Characteristic description. A description of a class of objects which uses descriptive facts that are true for all objects in the class and thus discriminates between objects in the class and all other objects. A characteristic description of a house is one which sets houses apart from all nonhouses.

Discriminant description. A description of a class given in the context of a fixed set of other classes. A fruit tree that bears apples is a discriminant description of an apple tree within the context of fruit trees.

Given specifications for object descriptions and the type of inductive assertions desired, INDUCE can discover and formulate general class descriptions of the objects. We give here only a brief outline of the basic steps followed when inducing a single-class description.

1. The user initially makes a determination of all descriptor types (attributes or relations) needed to describe the objects. This requires a specification of type, the domains, and if appropriate, the conditions of applicability. For example, in describing living cells, number_of_tails would apply only to cells with objects possessing tails. Next, each object is described using the given descriptors and the class to which it belongs. In addition, general rules and descriptors which apply for constructive induction are specified, and finally, the type of output description desired, the characteristic and/or discriminant descriptors.

2. Given the above information, INDUCE breaks down the object descriptions into new descriptors or complexes by dropping a single selector from each description; it then places them in a list. Each of these new structures represents a generalization of the object description (dropping condition rule). Clearly, some of these new descriptors will cover negative objects as well. Consequently, the descriptors are ordered giving highest rank to ones that cover the greatest number of positive objects and the fewest number of negative objects.

3. New descriptions are also created by applying inference rules to the original ones. The inference rules use constructive generalization, heuristics, and other infor-

mation to create the new descriptors. These are then added to the ranked list at their appropriate locations.

4. Each of the descriptions in the list is then tested for consistency and completeness. A description is consistent if it does not cover any negative object. It is complete if it covers all the (positive) objects. Those that pass the test are removed from the ranked list and put on a solutions list. Incomplete but consistent descriptions are put on a consistent list. Any descriptors remaining are specialized by appending selectors from the original list. These modified descriptions are tested for consistency and completeness and the process is repeated until predefined list size limits are exceeded or until all descriptors have been put on either the solutions or the consistent list.

5. Each of the descriptors on the consistent list is made more generic using generalizations such as climbing a generalization tree or closing an interval. The generalizations are then ranked and pruned using a Lexicographic Evaluation Function (LEF) and the best m of these are chosen as the description. The LEF uses criteria established by the user such as maximum examples covered, simplest descriptions (fewest terms), or user defined least cost. The final descriptions on the solutions list are the induced (generalized) descriptions which cover all the training instances. The following example will illustrate this process.

Assume the following three descriptions have been given for the object instances displayed in Figure 19.7.

- $$\begin{aligned} \exists o_1, o_2 \quad & [\text{color}(o_1) = \text{green}] [\text{shape}(o_1) = \text{sphere}] [\text{size}(o_1) = \text{large}] \\ & [\text{color}(o_2) = \text{red}] [\text{shape}(o_2) = \text{box}] [\text{size}(o_2) = \text{large}] \\ & [\text{supports}(o_2, o_1)] \\ \exists o_3, o_4 \quad & [\text{color}(o_3) = \text{red}] [\text{shape}(o_3) = \text{cylinder}] [\text{size}(o_3) = \text{small}] \\ & [\text{color}(o_4) = \text{red}] [\text{shape}(o_4) = \text{cube}] [\text{size}(o_4) = \text{large}] \\ & [\text{supports}(o_4, o_3)] \\ \exists o_5, o_6 \quad & [\text{color}(o_5) = \text{blue}] [\text{shape}(o_5) = \text{pyramid}] [\text{size}(o_5) = \text{small}] \\ & [\text{color}(o_6) = \text{red}] [\text{shape}(o_6) = \text{cylinder}] [\text{size}(o_6) = \text{large}] \\ & [\text{supports}(o_6, o_5)] \end{aligned}$$

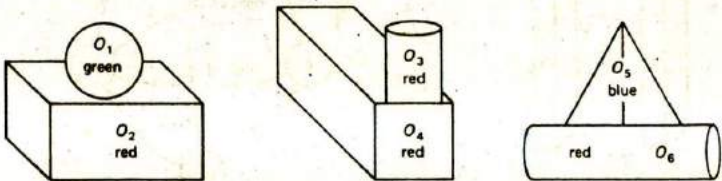


Figure 19.7 Three positive examples for training.

Using the procedure outlined above, INDUCE would discover the generalized description for the examples as "a red object supports another object," that is

$$\exists x,y [\text{color}(x) = \text{red}] [\text{supports}(x y)]$$

19.5 LEARNING STRUCTURAL CONCEPTS

Winston's Arch system was developed early in 1970 (Winston, 1975). This work has been noted as one of the most influential projects in recent AI research and has been cited as being responsible for stimulating renewed research in the area of machine learning.

The Arch system learns concepts in the form of associative network representations (Figure 19.8) much like the cluster network representations of the previous chapter. The Arch system, however, is not able to handle disjunctive descriptions.

Given positive and negative training examples like the ones in Figures 19.9 and 19.10, the system builds a generalized network description of an arch such as

```
(if (and (has_parts x (x1 x2 x3))
        (ako brick x1)
        (ako brick x2)
        (ako prism x3)
        (supports x1 x3)
        (supports x2 x3))
    (isa x arch))
```

Each training example is presented as a blocks world line drawing which is converted to a network representation. The first positive example is taken as the

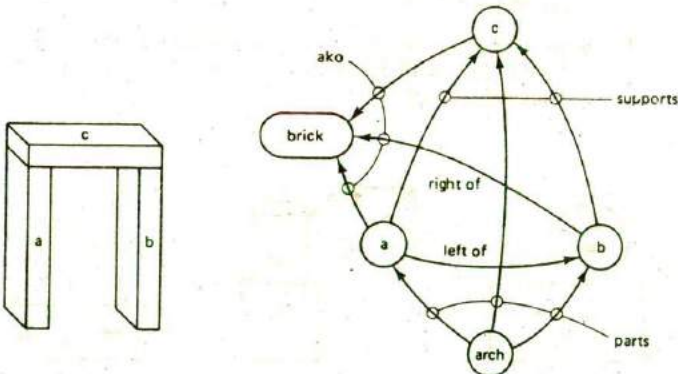


Figure 19.8 An arch and its network representation.

<code>((arch(nodes(a b c)</code>	<code>(arch(nodes(a b c)</code>	<code>(arch(nodes(a b c)</code>
<code>(links(ako a brick)</code>	<code>(links(ako a brick)</code>	<code>(links(ako a brick)</code>
<code>(ako b brick)</code>	<code>(ako b brick)</code>	<code>(ako b brick)</code>
<code>(ako c brick)</code>	<code>(ako c wedge)</code>	<code>(ako c prism)</code>
<code>(supports a c)</code>	<code>(supports a c)</code>	<code>(supports a c)</code>
<code>(supports b c)</code>	<code>(supports b c)</code>	<code>(supports b c)</code>
<code>(right_of b a)</code>	<code>(right_of b a)</code>	<code>(right_of b a)</code>
<code>(left_of a b)))</code>	<code>(left_of a b)))</code>	<code>(left_of a b)))</code>
Positive Example	Positive Example	Generalization

Figure 19.9 Generalized description for two positive examples.

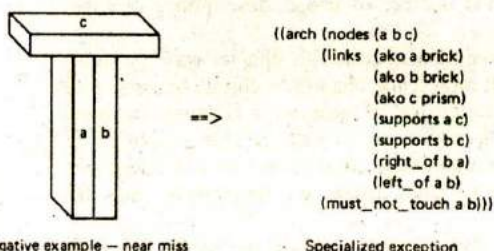


Figure 19.10 A nonarch and the resulting specialized representation.

initial concept description. The next example is then matched against this description using a graph-matching algorithm. This produces a common subgraph and a list of nodes and links which differ. The unmatched nodes and links are tagged with comments which are used to determine how the current description should be modified. If the new example is positive, the description is generalized (Figure 19.9) by either dropping nodes or links or replacing them with more generalized ones obtained from a hierarchical generalization tree. If the new example is a negative one, the description is specialized to exclude that example (Figure 19.10).

The negative examples are called *near misses*, since they differ from a positive example in only a single detail. Note the form of specialization used in Figure 19.10. This is an example of specialization by taking exception. The network representation for these exceptions are *must* and *must not* links to emphasize the fact that an arch must not have these features.

19.6 SUMMARY

Examples of four different inductive learning paradigms were presented in this chapter. In the first paradigm, the ID3 system, classifications were learned from a set of positive examples only. The examples were described as attribute values of objects. The classifications were learned in the form of discrimination tree. Once created, the ID3 system used the tree to classify new unknown objects. Attributes are selected

on the basis of the information Gain expected. This results in a minimal tree size. LEX, the second system described, learned heuristics to choose when certain operators should be used in symbolic integration problems. One of the interesting features of LEX is the use of a version space which bounds the set of plausible heuristics that are applicable in a given problem state. LEX uses a syntactic form of bias, its grammar, to limit the size of the hypothesis space. In LEX, generalizations are found by climbing a hierarchical description tree.

The third system considered, INDUCE, formed generalized descriptions of a class of objects in an extended form of predicate calculus. This system builds both attribute and structural types of descriptions. One weakness of this system, however, is the amount of processing required when creating descriptions by removing the selectors in all possible ways. When the number of object descriptions becomes large, the computation becomes excessive.

The fourth and final inductive learner described in this chapter was Winston's Arch. This system builds associative net representations of structural concepts. One of the unique aspects of this system is the use of near miss negative examples which differ from positive examples in only a single feature. This simplifies the learning process somewhat. This system is similar in some ways to the one described in the previous chapter. It is not able to build disjunctive descriptions like that system, however.

Similar features shared by all of these systems include the use of symbolic representations, and the methods of generalization and specialization. The principal differences among these paradigms are the forms of symbolic representation schemes used and the algorithms employed for generalizing and specializing.

EXERCISES

- 19.1. Derive the discrimination tree of Figure 19.2 using attributes arranged in the following order: height, weight, horned, and color.
- 19.2. Prove that the entropy H is maximum when $p_i = p_j$ for all $i, j = 1, 2, \dots, n$, where

$$H = -\sum_i p_i * \log_2 p_i$$

- 19.3. Plot the entropy as a function of p for the case $n = 2$; that is, plot H as p ranges from 0 to 1.
- 19.4. Describe how LEX generalizes from tan, cos, ln, and log to transc (transcendental).
- 19.5. Use the concepts related to version space bounding to illustrate how a system can learn the concept of a large circle from both positive and negative examples of objects described by the attributes shape (circle, square, triangle), size (small, medium, large), and color (red, blue, green).
- 19.6. What is the difference between a standard disjunction and an internal disjunction?
- 19.7. What is the significance of a cover, and how does it relate to a concept?

- 19.8. What steps, if any, are taken by INDUCE to avoid overgeneralization?
- 19.9. Compare the methods of induction used by ID3, LEX and INDUCE. Which of the methods is most efficient? Which is most robust? Which has greatest scope; that is, which is most domain independent?
- 19.10. Compare Winston's Arch learning system with the system developed by Iba (see Chapter 18). Which is most versatile?
- 19.11. What is the inductive leap used in inductive learning? Why is it potentially dangerous, but still useful? At what point can it be taken?

Analogical and Explanation-Based Learning

In this last chapter on autonomous acquisition, we investigate two of the most promising approaches to artificial learning, analogical and explanation-based learning. These approaches are both potentially powerful forms of learning, yet to date, they have received less attention than other methods. We expect this will change because of the great promise they both offer.

20.1 INTRODUCTION

Unlike inductive or similarity-based learning which is based on the observance of a number of training examples, analogical and explanation-based learning (EBL) can be accomplished with only a single example. In analogical learning, one example in the form of a known solution or a past experience is often sufficient knowledge for learning a new solution. In EBL, one positive training example is all that is needed to develop an explanation of a target concept. Of course, multiple examples may be used in both of these learning types, but in general, only a single example is required.

Analogies play an important role in our reasoning processes. We frequently explain or justify one phenomenon with another. A previous experience can often serve as a framework or pattern for a new, analogous experience. We use the

familiar experience as a guide in dealing with the new experience. And, since so many of our acts are near repetitions of previous acts, analogical learning has gained a prominent place in our learning processes.

EBL methods of learning differ from other methods in that the learned knowledge is valid knowledge. It is derived from a set of facts through a deductive reasoning process and, therefore, is justified, consistent knowledge. These EBL methods will most likely find use in conjunction with other learning paradigms where it is important to validate newly learned knowledge.

20.2 ANALOGICAL REASONING AND LEARNING

Analogies are similarities or likenesses between things otherwise different. Things which are similar in some respects tend to be similar in other respects. The things or participants in analogies are unlimited. They may be physical objects, concepts, problems and their solutions, plans, situations, episodes, and so forth.

Analogies play a dominant role in human reasoning and learning processes. Previously remembered experiences are transformed and extended to fit new unfamiliar situations. The old experiences provide scenarios or explanations which tend to match the new situation in some aspects and, therefore, offer the promise of suggesting a solution to the new situation. The old and new situations need not be in the same domain. Frequently, the two domains, the base and target domains, will be entirely different, but similarities between the relationships of objects remain strong, a likely consequence of the regularity of nature. For example, consider the solar planetary system and the Rutherford model of an atom or the flow of liquids in pipes and the flow of electricity in conducting wires. In the first case, both consist of a system of smaller bodies being attracted to and revolving around a more massive nucleus. In the second case, the flow behaviors are governed by the force of the sources, the resistance of the medium, and other properties of the respective flow in networks.

Analogical reasoning is probably the form of reasoning we are most dependent upon for all of our decisions and actions. Its use spans such mundane tasks as finding one's way home from work, to more complex tasks such as playing chess or writing a technical paper. As suggested by Carbonell and Minton (1983), this form of reasoning requires less cognitive effort than more formal types of reasoning, which could explain why analogical reasoning is so prevalent in human thought processes.

Analogies appear in different guises and at varied levels of abstraction. Simpler analogies are the word-object or geometric ones often found in SAT or GRE tests. They take the form

$A \sim B$

$A:B :: C:D$

(A is like B) or more generally
(A is to B as C is to D)

where one of the components is missing. For example, the type of word-object and geometrical analogies typically found in aptitude or GRE tests are given by

(a)	(b)
house : hut	(b) water : dam
tree : _____	_____ : battery
(c)	(d)
green : go	# 0 *
red : _____	0 *
	* # 0

Examples of more abstract analogies are the planetary system or atomic model noted above, the proof of a theorem based on a similar known proof, solving a new problem from knowledge of an old familiar problem solving technique, learning to play the card game of bridge from a knowledge of hearts, or producing a new algorithm for a program using previously learned programming examples and concepts.

Although applications of analogical methods have received less attention in AI than other methods, some important results have been published including (Burstein, 1986, Carbonell, 1983 and 1986, Greiner, 1988, Kling, 1971, McDermott, 1979, and Winston, 1980). Researchers in related fields have also made important contributions: cognitive science (Gentner, 1983), and psychology (Rumelhart and Norman, 1981). Several of the researchers in AI have produced working programs based on their models of the analogical process. We examine some of these models in the following section. In the remainder of this section we investigate the analogical reasoning process in some detail.

The Analogical Reasoning Process

Analogical reasoning was briefly described in Chapter 4 as a nonvalid form of inference. This follows from the fact that conclusions based on analogies are not necessarily logical entailments of the source knowledge. Analogy is a form of plausible reasoning. It is based on previously tested knowledge that bears a strong resemblance to the current situation.

There are five essential steps involved in the analogical learning process. A central step is the mapping process which is illustrated in Figure 20.1 and described below.

Analogical Learning Process

1. **Analogue recognition:** A new problem or situation is encountered and recognized as being similar to a previously encountered situation

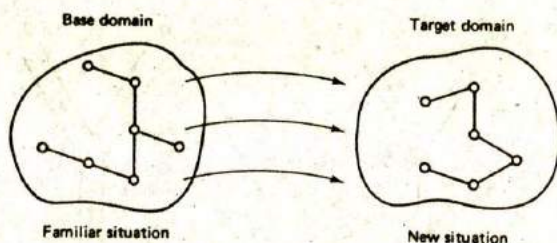


Figure 20.1 The analogical mapping process.

2. Access and recall: The similarity of the new problem to previously experienced ones serves as an index with which to access and recall one or more candidate experiences (analogues).
3. Selection and mapping: Relevant parts of the recalled experiences are selected for their similarities and mapped from the base to the target domain.
4. Extending the mapped experience: The newly mapped analogues are modified and extended to fit the target domain situation.
5. Validation and generalization: The newly formulated solution is validated for its applicability through some form of trial process (such as theorem provers or simulation). If the validation is supported, a generalized solution is formed which accounts for both the old and the new situations.

Implementation of the above process requires that several important issues be addressed. These issues are recurrent ones that appeared several times in other problems.

First, there is the question of representation. For analogical reasoning, it is desirable that knowledge be stored in a form that is easily accessed, retrieved, and evaluated for possible use as a candidate solution. This implies that self-contained, interrelated pieces of knowledge comprising a particular situation, episode, proof, plan, concept, and other unit of knowledge should be indexed and stored for simultaneous recall. Object attributes and relationships should be bound together with identifiers and other descriptive information for ease of use.

Second, it is desirable that an appropriate similarity measure be used to insure that only candidate analogues bearing a strong similarity to the new situation be considered.

Third, it is important that the mapping process from base to target domain be flexible enough to capture and import only the appropriate parts of the descriptions. The mapping should be able to transform objects, attributes, and predicates into corresponding ones which best meet the requirements of the target domain situation. For a general analogical system, this means that the transformation process should be dynamically adaptable to map between different domains and at different levels of abstraction.

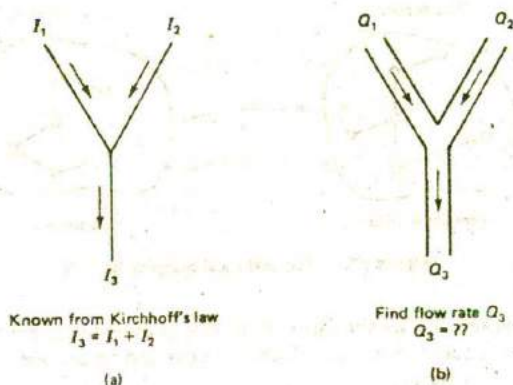


Figure 20.2 Analogical problem solving example.

Next, the newly created solution must be tested for its suitability. This test can be as informal as a simulated solution trial in the target domain or as formal as a deductive test of logical entailment.

Finally, having found an analogy and tested it successfully, the resultant episode should be generalized if possible and then summarized, encoded, indexed and stored for subsequent use in reasoning or learning.

A simple example will help to illustrate this process. Suppose we are given the problem of determining the flow rate of a fluid from a simple Y junction of pipes (Figure 20.2b). We are asked to determine the value of Q_3 given only knowledge of the flow rates Q_1 and Q_2 . A description of this unknown problem reminds us of a similar known problem, that of finding the flow rate of electrical current in a circuit junction. We recall the solution method to the electrical problem as being that based on Kirchhoff's current flow law, namely, that the sum of the currents at a junction is zero. We use this knowledge in an attempt to solve the hydraulic flow problem using the same principles; that is, we map the electrical flow solution to the hydraulic flow problem domain. This requires that corresponding objects, attributes and relations be suitably mapped from the electrical to the hydraulic domain. We then test the conjectured solution in some way.

In the reminding process, we may alternatively be given a direct hint from a teacher that the hydraulic flow problem is like the electrical flow problem. Otherwise, we must infer this likeness in some way and proceed with the conjecture that they are alike, justifying the likeness on the basis of the consistency of nature or some other means.

Next, we examine some representative examples of analogical learning systems.

20.3 EXAMPLES OF ANALOGICAL LEARNING SYSTEMS

Winston's System

Patrick Winston (1980) developed programs that reason about relationships, motives, and consequent actions that occur among people. Using relationships and acts of actors in one story (such as *Macbeth*) the program was able to demonstrate that analogous results occurred in different stories (such as *Hamlet*) when there were similarities among the relationships and motives of the second group of characters. The programs could also learn through the analogical reasoning process. For example, when a teacher declared that voltage, current, and resistance relationships were like those of water pressure, flow, and pipe resistance, the system was able to learn basic results in electrical circuits and related laws such as Ohm's law (the opposite of the learning problem described above).

The analogical mapping and learning process for this example is illustrated in Figure 20.3. The items in the figure labeled as voltage-value-3, current-value-3, and resistance-value-3 represent specific values of voltage, current, and resistance, respectively.

The important features of Winston's system can be summarized as follows:

1. Knowledge representation: Winston's system used frame structures as part of the Frame Representation Language (FRL) developed by Roberts and Goldstein (1977). Slots within the frames were given special meanings, such as AKO, appears-in, and the like. Individual frames were linked together in a network for easy access to related items.

2. Recall of analogous situations: When presented with a current situation, candidate analogues were retrieved from memory using an hierarchical indexing scheme. This was accomplished by storing a situations (frame) name in the slots of all object frames that appeared in the situation. For example, in the Cinderella story, the prince is one of the central parts. Therefore, prince would be used as a node in a hierarchical tree structure with sublinks

prince→AKO-man→AKO-person

and the Cinderella label CI would be stored in an appears-in slot of the frames belonging to the prince, the man, and the person. These slots were always searched as part of the recall reminding process when looking for candidate analogues.

3. Similarity matching: In selecting the best of the known situations during the reminding process described above, a similarity matching score is computed for each of the recalled candidates. A score is computed for all slot pairings between two frames, and the pairing having the highest score is selected as the proper analogue. The scoring takes into account the number of values that match between slots as

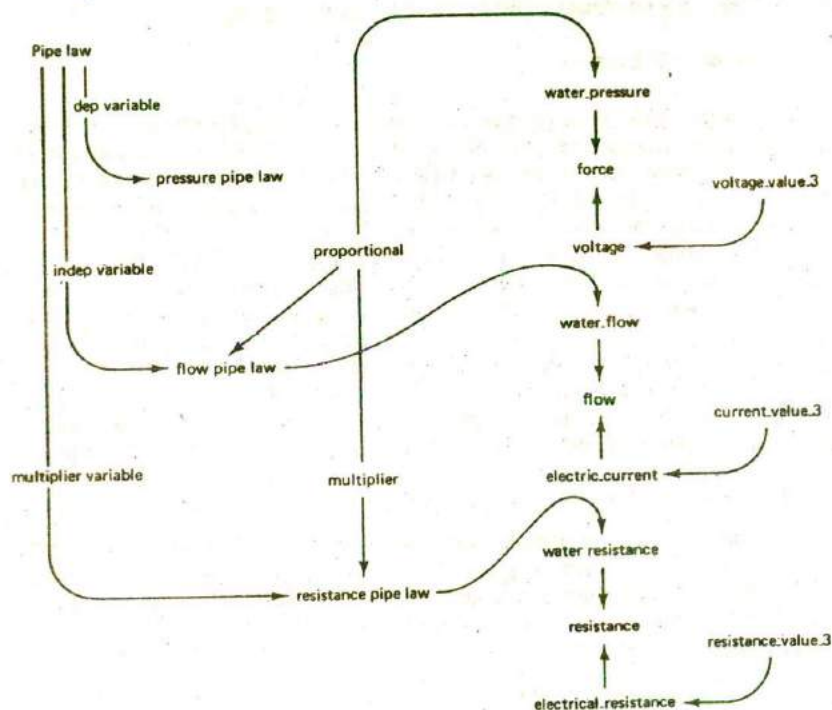


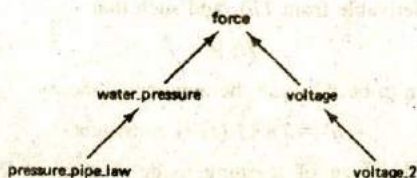
Figure 20.3 Learning electrical laws from hydraulics.

well as matching relationships found between like parts having causal relations as noted in comment fields.

4. Mapping base-to-target situations: The base-to-target analogue mapping process used in this system depends on the similarity of parts between base and target domains and role links that can be established between the two. For example, when both base and target situations share the same domain, such as water-flow in pipes, parts between the two are easily matched for equality. A specific pipe, *pipe1*, is matched with a general pipe, *pipe#*, specific water-flow, with general flow, and so forth. The relationships from the general case are then mapped directly to the specific case without change.

In cases where base and target are different domains, the mapping is more difficult as parts, in general, will differ. Before mapping is attempted, links are established between corresponding parts. For example, if the two domains are water-flow and electricity-flow (it is known the two are alike), determining the electrical

resistance in a circuit is achieved by mapping the pipe-water-flow laws to the electrical domain. Before mapping, a link between the pressure and voltage laws and the water-flow and current-flow laws is established. The link is determined from a hierarchical tree connecting the two parts, where voltage-2 is a specific voltage.



In all of the above components, matching plays a dominant role. First, in recall, matching is used to determine candidate situations. Second, matching is used for scoring to select the best analogue. Finally, matching is used to determine how parts and relationships are mapped from base to target domains.

Greiner's NLAG System

Russell Greiner (1988) developed an analogical learning system he called NLAG using Stanford University's MRS logic development language. The system requires three inputs and produces a single output. The inputs are an impoverished theory (a theory lacking in knowledge with which to solve a given problem), an analogical hint, and the given problem for solution. The output from the system is a solution conjecture for the problem based on the analogical hint.

Using Greiner's notation, the process can be described as follows. The analogical inference process described here is represented by the operator $\vdash$. This operator takes three inputs.

1. A finite collection of consistent propositions (rules, facts, and the like) called a theory (Th).
2. An analogical hint about the source and target analogues A and B respectively, written $A \sim B$ (A is like B). Here $A = \{a_1, \dots, a_n\}$ and $B = \{b_1, \dots, b_n\}$ are sets of arbitrary formulae or knowledge related to some problem situations.
3. A problem to be solved in the target domain (target problem) denoted as PT .

The output from the system is a set of new propositions or conjectures $\phi(A)$ related to the set B that can be used to solve PT .

The above process can be summarized as follows:

$$Th, A \sim B \vdash_{PT} \phi(A)$$

The NLAG system was designed to perform useful, analogical learning. We define the *learning* part here as the process of determining or creating one or more formulae f which are not initially in the deductive closure of Th , that is, such that

$$Th \not\models f \text{ holds,}$$

(so f is not initially derivable from Th), and such that

$$Th \not\models \neg f,$$

that is, f is not known to be false; so the augmented theory

$$Th' = Th \cup \{f\} \text{ is consistent.}$$

We specialize this type of learning to be *analogical learning* by requiring that the process use the operator $\vdash$ and have as inputs both the theory Th as well as the hint $A \sim B$. Also, the new formulae f should be about the target analogue A ($f = \phi(A)$ where ϕ is some set of arbitrary formulae). The source analogue B should also satisfy the analogy formulae. Thus, we must also have $Th \models \phi(B)$.

Finally, we specialize the definition further to *useful analogical learning* by requiring that the conjectured formulae $\phi(A)$ returned by the system be limited to those formulae that are useful in the sense that the ϕ , together with Th , can solve PT , that is

$$Th \cup \{\phi(A)\} \models PT.$$

This definition of useful analogical learning is summarized in Figure 20.4 where we have named the conditions described in the above definition as unknown, consistent, common, and useful.

These notions are illustrated in terms of our earlier example for the hydraulics or electrical flow problem (Figure 20.2). Thus, B here corresponds to the known theory Th that given an electrical Y-junction as in Figure 20.2a, then $I_3 = I_1 + I_2$. The problem PT is to find Q_3 , the fluid flow rate in a similar Y-junction of pipes when Q_1 and Q_2 are known. The analogical hint, $A \sim B$, is that the hydraulics flow problem is like the electrical current flow problem. The useful formula needed to solve the problem is of course $Q_3 = Q_1 + Q_2$. A more complex example would, of course, require more than a single formula.

Since many analogies may satisfy the theory and the analogical hint, it is necessary to restrict the analogies considered to those which are useful. For this, NLAG uses heuristics to select only those formulae which are likely to be useful.

One heuristic used by the system is based on the idea that relationships in

	$Th, A \sim B \vdash_{PT} \phi(A)$	
where	UNKNOWN:	$Th \not\models \phi(A)$
	CONSISTENT:	$Th \not\models \neg \phi(A)$
	COMMON:	$Th \models \phi(B)$
	USEFUL:	$Th \cup \{\phi(A)\} \models_{PT}$

Figure 20.4 Summary of useful analogical inference.

one domain should hold in other, similar domains (in two domains governed by physical laws). In our example, this is the law related to zero flow rates into a junction and the corresponding reusable zero-sum formula for general, physical flows. Thus, only those uninstantiated abstract formulas found in the base analogue would be permitted as conjectures in the target analogue solution. Furthermore, this same heuristic requires that the selected formulae be atomic; formulae of the form $f(a_1, a_2, \dots, a_k)$ are permitted, whereas formulae containing multiple conjunctive or disjunctive terms such as

$$f_1(a,b) \ \& \ f_2(b,c) \ \vee \ f_3(a,c,d,e)$$

are not permitted. These formulae must also satisfy the usefulness condition (Figure 20.4).

A heuristic which helps to further prune the abstract solutions described above uses the target domain problem *PT*, to suggest a related query in the source domain. The corresponding source domain query *PS* is then used in turn to select only those formulae which are both abstractions (as selected above) and are also relevant. For example, the target query *PT* = "Find the flowrate in the given pipe structure" is used to find the analogous source domain query *PS* = "Find the current in the given electrical structure." This query is then used to determine which facts are used to solve *PS*. These facts are then used as a guide in selecting facts (formulae) for the target domain to solve *PT*.

Other system heuristics require that formulae must all come from the same domain, that more general abstractions be preferred over less general ones, and that for a given abstraction, instances which require the fewest conjectures be chosen. In general, these heuristics are all based on choosing only enough new information about the target analogue to solve *PT* and then stop.

Carbonell's Systems

Carbonell developed two analogical systems for problem solving, each based on a different perception of the analogical process (1983 and 1986). The first system was based on what he termed transformational analogy and the second on derivational analogy. The major differences between the two methods lie in the amount of details remembered and stored for situations such as problem solution traces and the methods used in the base-to-target domain transformational process.

Both methods used a knowledge representation and memory indexing and recall scheme similar to the memory organization packets or MOPs of Roger Schank (Chapter 11). Both methods also essentially followed the five-step analogical learning process outlined in the previous section. The main differences between the two methods can be summarized as follows:

Transformational Analogy. Problem solving with this approach is based on the use of means-end analysis (MEA) as described in Chapter 9. Known problem solutions are indexed and stored for later retrieval. The solutions are stored as an

initial state, a goal state, and a sequence of actions (operators) which, when applied to the initial and intermediate states, result in a transformation to the goal state. When a new problem is encountered, it is matched against potentially relevant known ones using a suitable similarity measurement. The partial match producing the highest similarity measure is transformed to satisfy the requirements of the new problem. In finding a solution to the new problem using the known mapped solution, it is often necessary to disturb some states to find operators which reduced the current-to-goal state differences. With this method, the focus is on the sequence of actions in a given solution, and not on the derivation process itself.

Derivational Analogy. This approach requires the storage of much more information than the transformational approach. It is intended that the analogies here be made more on the basis of the complete reasoning process than on the sequence of solution states for the past problem solutions. In solving newly encountered problems, a plan is formulated using the complete solution trace of the analogue. This plan includes a list of all subgoals derived, alternative solution paths generated and those rejected, each step of the solution process and the decisions made at each step, pointers to knowledge which proved to be useful in the solution, and detailed failure information when a solution cannot be found. This plan is modified to fit the new problem domain. If it cannot produce a solution, other methods are applied such as MEA.

Both of the systems developed by Carbonell are useful as analogical research tools. And both worked reasonably well in more than one domain.

20.4 EXPLANATION-BASED LEARNING

One of the most active areas of learning research to emerge during the early 1980s is explanation-based learning (also known as explanation-based generalization). This is essentially a form of deductive generalization. It has been successfully demonstrated for the general area of concept learning.

In EBL, four kinds of information must be known to the learner in advance.

1. A formal statement of the goal concept to be learned,
2. At least one positive training example of the concept,
3. Domain theory, which relates to the concept and the training example, and
4. Criteria for concept operationality.

The notion of operationality in item 4 requires some explanation. We say a procedure is operational if, for some agent and task, the procedure can be applied by the agent to solve the task. Thus, if the concept to be learned is one related to the recognition of pencils, operational criteria in this case would be given as a structural definition which could serve to uniquely identify pencils. If the concept is related to the function or use of pencils, the operational criteria would be given as a functional definition.

The objective of EBL is to formulate a generalized explanation of the goal concept. The explanation should prove how the training example satisfies the concept definition in terms of the domain theory.

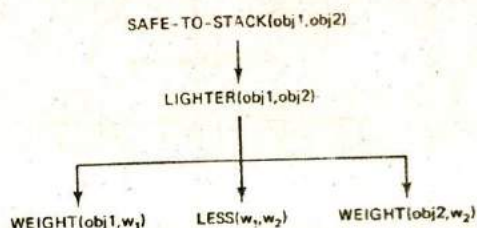
The EBL method constructs an explanation of the concept based on the given example in terms of the domain theory. The explanation is a deductive proof of how the example satisfies the goal concept definition. Once an explanation has been developed, it is generalized by regressing the goal concept through the explanation structure. We illustrate these ideas with an example given by Mitchell, Keller, and Kedar-Cabelli (1986).

The concept to be learned in this example is the conditions under which it is safe to stack one item on top of another item. The training example is given as a description of a pair of objects; one object is a box which is stacked on top of the other object, an end table. The domain knowledge needed by the system consists of some structural attributes and relationships between physical objects such as volume, density, weight, lighter-than, fragile, and so on. The operational criteria in this case requires that the concept definition be expressed in terms of the predicates used to describe the example. The objective of the learning process here is to develop a generalization of the training example that is a sufficient concept definition for the goal concept which satisfies the operability criteria. These definitions are summarized in Figure 20.5.

In carrying out the EBL method, the system must first construct an explanation of how the training example satisfies the goal concept. This requires that it select those attributes and relations which are relevant and which satisfy the operational criteria. From the goal concept definition it is seen that it is safe-to-stack an object x on an object y if and only if either y is not fragile or x is lighter than y . A search through the domain theory for the predicate FRAGILE fails; therefore, a search is then made for the predicate LIGHTER. This predicate is found in the rule

$$\text{WEIGHT}(p_1, v_1) \ \& \ \text{WEIGHT}(p_2, w_2) \ \& \ \text{LESS}(w_1, w_2) \rightarrow \text{LIGHTER}(p_1, p_2)$$

Matching obj1 to x (to p_1 in the above rule) and obj2 to y (to p_2 in the above rule) we can form the following partial explanation for safe-to-stack.



Continuing with the explanation process, we find rules relating the weights with volume and density and to an endtable. After instantiation of these terms, we

Known:

Concept Definition:

Pairs of objects $\langle x, y \rangle$ are SAFE-TO-STACK when
 $\text{SAFE-TO-STACK}(x, y) \leftrightarrow \sim \text{FRAGILE}(y) \vee \text{LIGHTER}(x, y)$.

Training Example:

ON(obj1, obj2)
 ISA(obj1, box)
 ISA(obj2, endtable)
 COLOR(obj1, red)
 COLOR(obj2, blue)
 VOLUME(obj1, 1)
 DENSITY(obj1, 0.1)

Domain Theory:

$\text{VOLUME}(p_1, v_1) \ \& \ \text{DENSITY}(p_1, d_1) \rightarrow \text{WEIGHT}(p_1, v_1 * d_1)$
 $\text{WEIGHT}(p_1, w_1) \ \& \ \text{WEIGHT}(p_2, w_2) \ \& \ \text{LESS}(w_1, w_2) \rightarrow$
 $\text{LIGHTER}(p_1, p_2)$

$\text{ISA}(p_1, \text{endtable}) \rightarrow \text{WEIGHT}(p_1, 5)$ (i.e. a default)
 $\text{LESS}(0.1, 5)$

Operationality Criteria:

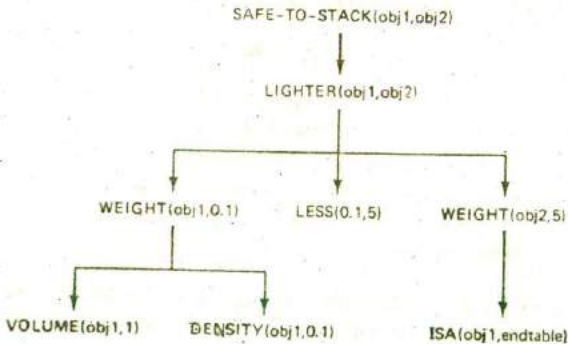
The learned concept should be expressed in terms of the same predicates used to describe the example, i.e. COLOR, DENSITY, VOLUME, etc. or simple predicates from the domain theory (e.g. LESS).

Determine:

A generalization of the training example that is a sufficient goal concept definition which satisfies the operationality criteria.

Figure 20.5 Safe-to-stack example for EBL.

are led to the complete explanation tree structure given below in which the root terms are seen to satisfy the operationality criteria.



The next step in developing the final target concept is to generalize the above structure by regressing or back-propagating formulae through rules in the above structure step by step, beginning with the top expression SAFE-TO-STACK and regressing SAFE-TO-STACK(x,y) through the goal rule (the FRAGILE disjunct is omitted since it was not used). LIGHTER(p_1,p_2) $\rightarrow$ SAFE-TO-STACK(p_1,p_2) yields the term LIGHTER(x,y) as a sufficient condition for inferring SAFE-TO-STACK(x,y). In a similar way LIGHTER(x,y) is regressed back through the next step to yield WEIGHT(x,w_1) & WEIGHT(y,w_2) & LESS(w_1,w_2). This expression is then regressed through the final steps of the explanation structure to yield the following generalized, operational, definition of the safe-to-stack concept.

VOLUME(x,v_1)
 & DENSITY(x,d_1)
 & LESS($v_1*d_1,5$)
 & ISA($y, \text{endtable}$) $\rightarrow$ SAFE-TO-STACK(x,y)

The complete regression process is summarized in Figure 20.6 where the underlined expressions are the results of the regression steps and the substitutions are as noted within the braces.

In summary, the process described above produces a justified generalization of a single training example as a learned concept. It does this in a two step process. The first step creates an explanation that contains only relevant predicates. The second step uses the explanation structure to establish constraints on the predicate values that are sufficient for the explanation to apply in general. This differs from inductive learning in that a single training example is adequate to learn a valid description of the concept. Of course there is a trade-off here. The EBL method also requires appropriate domain knowledge as well as a definition of the target concept.

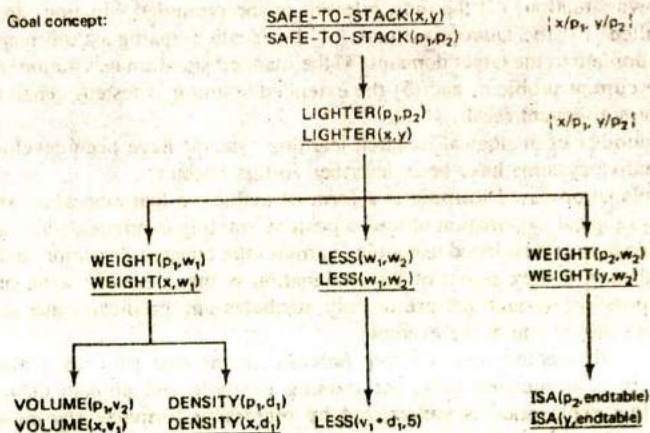


Figure 20.6 Generalization by regression through the explanation structure.

There is an apparent paradox in the EBL method in that it may appear that no actual learning takes place since the system must be given a definition of the very concept to be learned! The answer to this dilemma is that a broader, generalized, and more useable definition is being learned. With the EBL method, existing knowledge is being transformed to a more useful form. And, the learned concept applies to a broader class than the supplied definition. This newly learned concept is a valid definition since it has been logically justified through the explanation process. The same claim cannot be made of other nondeductive learning techniques like inductive or analogical learning.

The notion of operationality used in EBL systems should depend on the purpose of the learning system. As such, it should be treated as a dynamic property. For example, a robot may need to learn the concept of a pencil in order to recognize pencils. In this case, operationality should be interpreted in terms of the structural properties of the pencil. On the other hand, if the purpose of learning the pencil concept relates to design, the robot would be better served with a functional definition of a pencil. Keller (1988) discusses the operationality problem and its application in a program called MetaLEX (a successor to LEX described in Chapter 19).

20.5 SUMMARY

We have described two of the more promising approaches to machine learning, the analogical and explanation-based learning paradigms. These methods, unlike similarity-based methods, are capable of creating new knowledge from a single training example. Both methods offer great potential as autonomous learning methods.

Learning by analogy requires that similar, known experiences be available for use in explaining or solving newly encountered experiences. The complete process can be described in five steps: (1) a newly encountered situation serves as a reminder of a known situation, (2) the most relevant of the reminded situations are accessed and recalled, (3) the appropriate parts of the recalled analogues are mapped from the base domain to the target domain, (4) the mapped situation or solution is extended to fit the current problem, and (5) the extended solution is tested, generalized, and stored for subsequent recall.

A number of analogical research learning systems have been developed. Four representative systems have been described in this chapter.

Explanation-based learning is a form of deductive learning where the learner develops a logical explanation of how a positive training example defines a concept. The explanation is developed using the example, the concept definition, and relevant domain theory. A key aspect of the explanation is that it satisfy some operational criteria, possibly through the use of only attributes and predicates that are used in the domain theory and/or the example.

The EBL method is a two step process. In the first step, an explanation of the concept is formulated using the training example and domain theory. In the second, this explanation is generalized by regressing formulae step-by-step back

through rules in the explanation structure. The final concept, if successful, is a definition which both satisfies the operationality criteria and is also a valid definition of the concept.

EXERCISES

- 20.1. Describe two examples of analogical learning you have experienced recently.
- 20.2. Why is it that things similar in some ways tend to be similar in other ways?
- 20.3. Consult a good dictionary and determine the differences between the definitions of analogies, metaphors, and similes.
- 20.4. Make up three new analogies like the examples given in Section 20.2.
- 20.5. Relate each of the five steps followed in analogical learning to the following example: Riding a motorcycle is like riding a bicycle with an engine in it.
- 20.6. Compare the analogical system of Winston to that of Greiner. In what ways do they differ?
- 20.7. What appears to be more important in mapping from base to target domain, object attributes, object relationships, or both? Give examples to support your conclusions.
- 20.8. What are the main differences between Carbonell's transformational and derivational systems?
- 20.8. Define operationality as it applies to explanation-based learning and give an example of it as applied to some task.
- 20.10. Explain why each of the four kinds of information (concept definition, positive training example, domain theory, and operational criteria) is needed in EBL.
- 20.11. If the end table in the Safe-to-Stack example had not been given a default weight value, what additional steps would be required in the explanation to complete the tree structure?
- 20.12. What is the purpose of the regression process in EBL?
- 20.13. Work out a complete explanation for the concept safe to cross the street. This requires domain theory about traffic lights, and traffic, a positive example of a safe crossing, and operational criteria.
- 20.14. The learning methods described in Part V have mostly been single paradigm methods, yet we undoubtedly use combined learning for much of our knowledge learning. Describe how analogical learning could be combined with EBL as well as inductive learning to provide a more comprehensive form of learning.